

RAFM

This is a manual for the use of RAFM software. RAFM stands for R for admixture F model and has been published by Karhunen and Ovaskainen (2012). The idea of the admixture F model is to model the random drift in the allele frequencies of neutral DNA. More specifically, RAFM calculates a unique *coancestry coefficient* for each pair of populations. A coancestry coefficient is slightly different from the more commonly used F_{ST} , but it turns out that F_{ST} can be calculated from the set of coancestry coefficients. For more details, see Karhunen and Ovaskainen (2012).

This manual is intended to help you to use RAFM. It is written in a style different from the official R documentation, as the standard documentation is not a very informative format in this case. We assume that you have successfully unzipped the files in the RAFM.zip and you have set your R in the same working directory.

First, the data

Type

```
load("shrews.rda")
load("sticklebacks.rda")
load("specimen.rda")
```

`shrews.rda` and `sticklebacks.rda` involve data from *Sorex araneus* and *Pungitius pungitius*, respectively (see Karhunen and Ovaskainen 2012). However, we will work with the artificial data set `specimen`, as it is faster to run. Type

```
specimen[1:5, 1:5]
```

You should get

	subpop	loc1.1	loc1.2	loc2.1	loc2.2
1	1	NA	NA	2	3
2	1	4	1	3	4
3	1	4	1	2	5
4	1	1	4	3	2
5	1	1	1	2	5

That is, a series of subpopulation codes in column 1 and a set of oddly labeled alleles in columns 2+. *This is how you should format your data for use with RAFM.* Note that the individuals are assumed diploid, and consequently, there are two copies of each locus (e.g. `loc2.1` and `loc2.2`.)

Then, the program

Type

```
source("RAFM.r")
```

Type

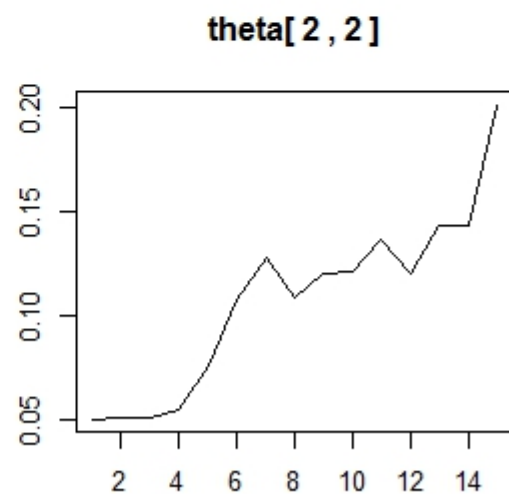
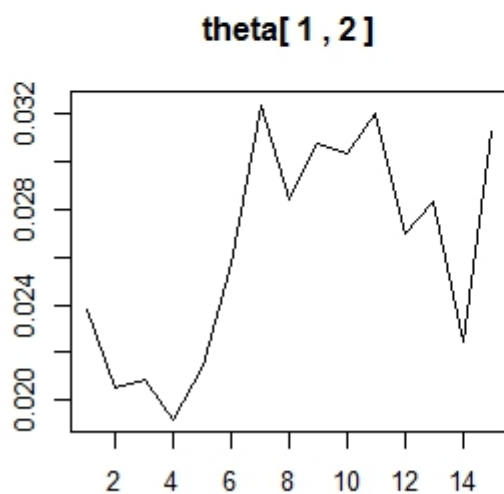
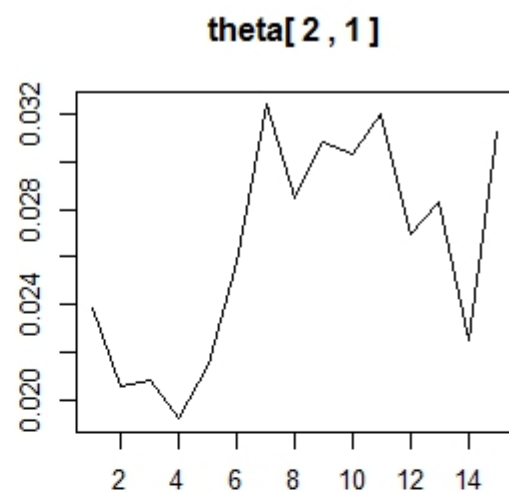
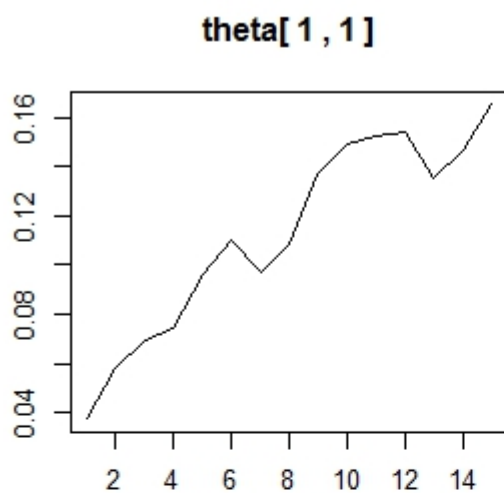
```
afm <- do.all(specimen, 50, 20, 2)
```

You should get

```
[1] posteriors written
```

```
[1] Drawing trace of coancestry matrix theta, assess convergence.
```

and



These are the Monte Carlo traces of the coancestry coefficients. We immediately notice that the Markov chains have not mixed well enough. This is because we have used a total sample size of 50. Type

```
head(do.all)
```

You should get

```
1 function (dat, nMC, burnin, thinning, eps = 7, priq = 1, pria = c(1,
2     2), prik = 1)
3 {
4     primary <- AFM(dat, nMC, burnin, thinning, eps = 7, priq = 1,
5         pria = c(1, 2), prik = 1)
6     kap <- primary[[1]]
```

We notice that the first argument of `do.all` is the data, the second one is the total sample size, the third one is the number of burn-in iterations, and the fourth one is the degree of thinning. So `do.all(specimen, 50, 20, 2)` means: analyze object `specimen` with total sample size 50, burn-in 20 and taking every second iteration. You can rerun `specimen` with a larger sample size, if you like.

Why `do.all`? `do.all` first executes `AFM` which is the MCMC estimation routine. Then it does a series of post-processing tricks, calculating the trace of F_{ST} and coancestry from the traces of primary parameters. It makes sense to wrap these to `do.all`, as to our experience, users find the parameter transformations rather annoying.

Making use of the results

Type

```
names(afm)
```

You should get

```
[1] "theta" "fst"  "kappa" "alpha"
```

The first one is the posterior of coancestry coefficients. The second one is the posterior of F_{ST} . The third one is the posterior of the *lineage loadings* and the fourth one is the posterior of the *drift parameters*. All of them are explained in Karhunen and Ovaskainen (2012). Now, type

```
summary(afm$fst)
```

You should get

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.02047 0.05607 0.08520 0.08610 0.11360 0.15670
```

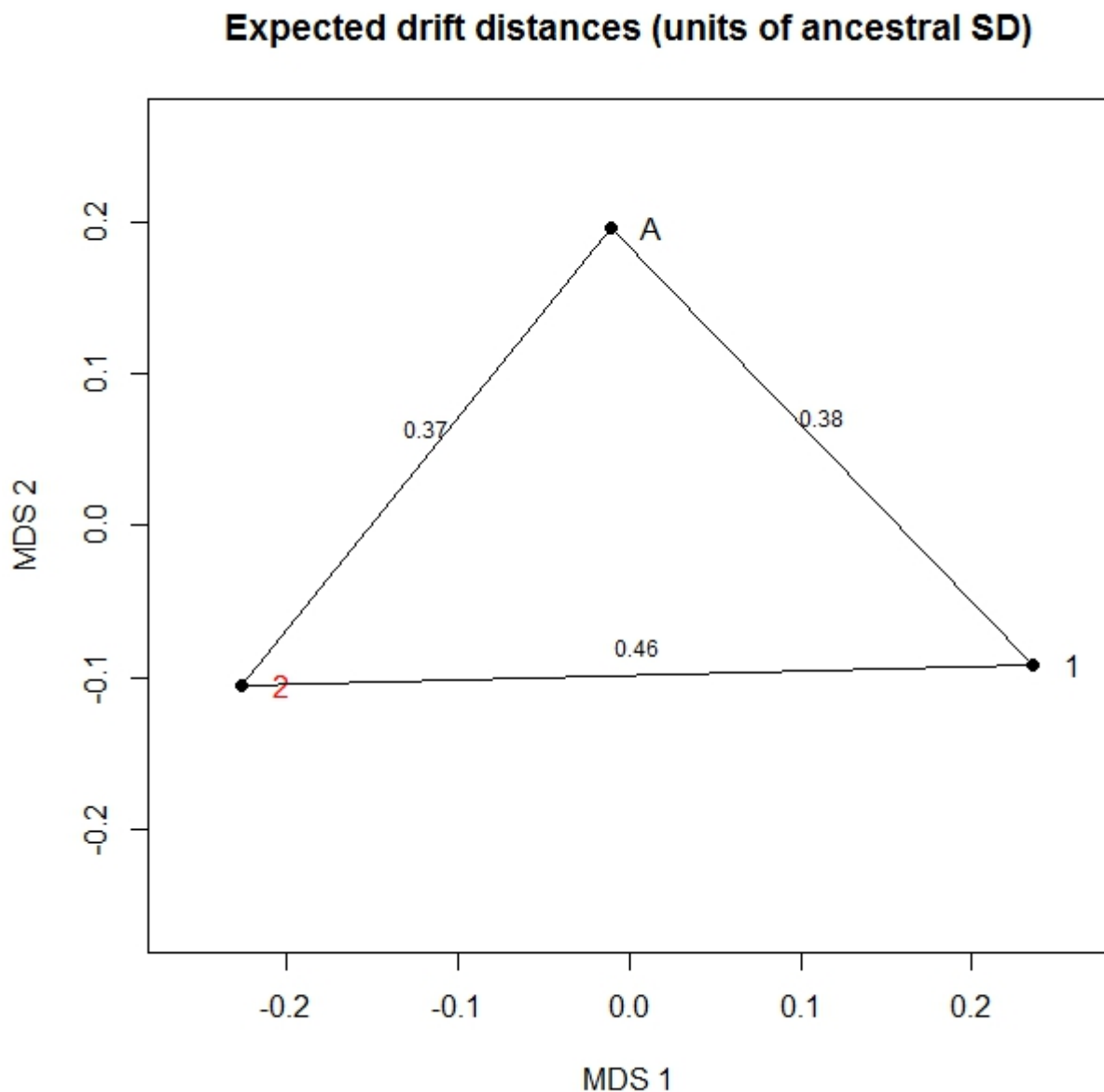
or something like that. (The numerical values change every time you run the program, if you have used an MC sample size as small as 50!) This is your posterior estimate of F_{ST} .

Now type

```
windows()
```

```
viz.theta(afm$theta)
```

You should get



or something like that, again depending on the Monte Carlo error. This is a pictorial representation of your coancestry matrix. A represents the hypothetical ancestral population, and other vertices represent the subpopulations. The coancestry matrix is a genuinely high-dimensional object, but now it has been forced into a planar graph by multidimensional scaling (MDS). The graph will become rather fancy, if you use a larger number of subpopulations.

The units of measurement are rather curious here. They are *standard deviations of a randomly drifting quantitative trait*. Namely, you can infer from the coancestry matrix, how much a trait mean should drift (see Ovaskainen et al. 2011). However, this topic is more in the domain of *driftsel* analysis. We will finish this section by looking at the posterior of your coancestry matrix itself. Type

```
dim(afm$theta)
```

You should get

```
[1] 2 2 15
```

The first dimensions (2 and 2) are the dimension of your coancestry matrix. The last dimension (15) is the size of your Monte Carlo sample. (This is 15, not 50, as the program has removed the burn-in and thinned the remaining chain.) Now type

```
summary(afm$theta[1,1,])
```

You should get

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.03744 0.08508 0.11000 0.11260 0.14760 0.16520
```

This is the posterior estimate of within-population coancestry in subpopulation 1. In analogy, `summary(afm$theta[1,2,])` would be the posterior estimate of the between-population coancestry, and `summary(afm$theta[1,2,])` would be the posterior estimate of the within-population coancestry in subpopulation 2. Of course, in scientific literature it is often customary to report 95 % credible intervals and such. To that end, you should be able to manipulate the Markov chains `afm$theta` and `afm$fst` by hand. As a first aid,

```
quantile(afm$theta[1,1,], 0.05)
```

gives the 5 % quantile of the respective within-population coancestry.

Custom analyses

Arguably, this manual presents the bare minimum you need to know to estimate coancestries by using RAFM. It would be of interest to adapt the program for different data types, to try different priors etc. Unfortunately, RAFM is no longer actively maintained, and we are slow to answer individual queries. However, we offer you the source code (`RAFM.r`) and the standard documentation (`RAFM_manual_old.pdf`). By using these two files, a skilled statistician should be able to further develop RAFM.

Known anomalies

Sometimes RAFM just crashes and gives an `Error`. This is most likely a result of numerical infinites, division by zero and the like in the MCMC function `AFM` (which `do.all` calls). These in turn result when the full conditional distributions of the allele frequencies go more extreme. These errors seem to occur more often for more polymorphic loci and for more differentiated subpopulations. So, as a first-aid solution you may want to consider removing some locus/population from your analysis. Unfortunately, not much more can be done without expert knowledge in numerical programming. (We have tried to remove these errors and catch them the best we can, but some still remain. To remove all of them would very probably require re-coding the [truncated] Dirichlet distributions and all the associated transcendental functions.)

References

Karhunen M and Ovaskainen O (2012): Estimating population-level coancestry coefficients by an admixture F model. *Genetics* 192(2): 609-617.

Ovaskainen O, Karhunen M, Zheng C, Arias JMC, Merilä J and Spencer HG (2011): A new method to uncover signatures of divergent and stabilizing selection in quantitative traits. *Genetics* 189(2): 621-632.